

Catching a wrong number that looked right

A worked example from the Merchant Sales & Voucher Intelligence build: how the gold layer is validated, and the defect that validation caught.

The short version

A sales target total was overstated by **R984,046**. Nothing was broken: the date table was contiguous, had no gaps, the correct row count, and every foreign key resolved. All 132 dbt tests passed. The table was structurally perfect and was answering a slightly different question than the one being asked.

What the data actually looks like

Sales and support activity end on **31 July 2026**. Vouchers sold before that date continue to be redeemed afterwards, with the last redemption on **20 August 2026**. Two different endings, 20 days apart, and the model needs both.

A date dimension must **span** to the last redemption, or a late redemption has no row to join to and silently drops out of every total. But the **reporting window** has to stop at the last sale, because there are no sales after it. Span and window are different things. Deriving both from one maximum conflates them.

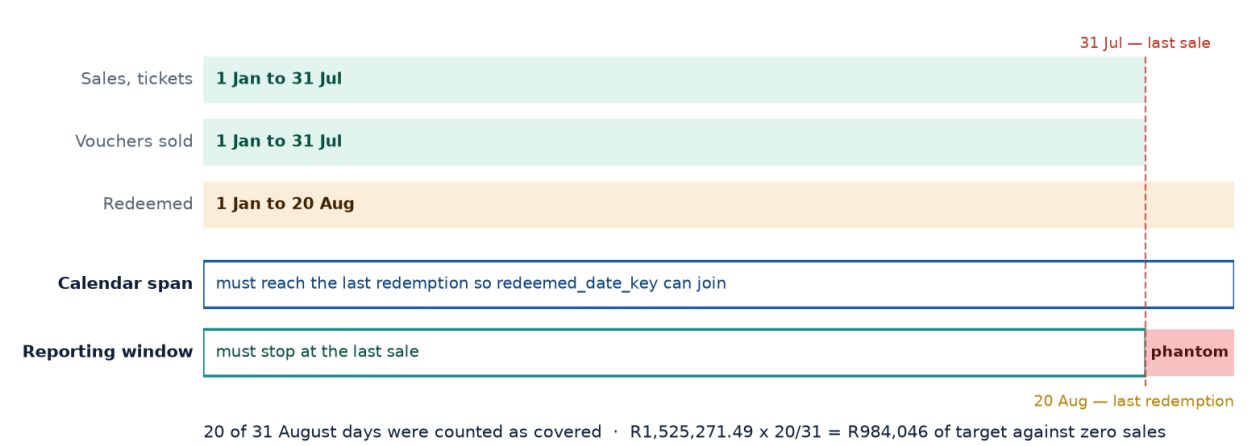


Figure 1 — the calendar must reach the last redemption; the reporting window must not.

How the wrong number arose

Monthly sales targets are pro-rated by the days the reporting window covers, so a part-month never shows a false shortfall. With the window running to 20 Aug, August was treated as **20 of 31 days covered**. Target is charged for those days. Sales for them are zero, because sales ended in July.

All-merchant monthly target	R1,525,271.49
August days wrongly counted	20 of 31
Pro-rated share	x 20/31
Target overstated by	R984,046
Correct total target	R10,676,900.43

What caught it

The gold layer is built **twice**, by two independent implementations — once in pandas, once in SQL executed by dbt. A reconciliation script compares every headline figure at a R0.01 tolerance and stops the build on any disagreement.

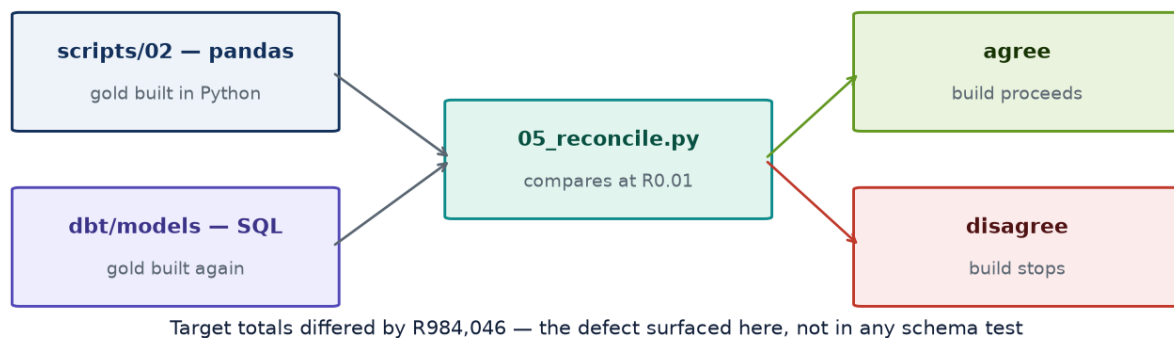


Figure 2 — two implementations, one answer. A disagreement is the signal.

This is the class of defect that schema tests cannot reach. A uniqueness test asks whether a table is well-formed; it cannot ask whether the period is the right period. Two independent implementations of the same business rule can both be well-formed and still disagree — and when they do, one of them is wrong.

The same gate caught a second defect: pandas `rank(pct=True)` returns `rank/n`, while SQL `PERCENT_RANK()` returns `(rank-1)/(n-1)`. Different statistics, both plausible, and merchant Health Score moved by up to 9.7 points depending on which was used.

The fix

One value. In the bounds CTE, the redemption branch contributes to the calendar maximum but passes NULL to the activity maximum, so it extends the span without moving the window:

```
-- Redemption tail extends the calendar but must NOT extend the reporting window
select min(redeemed_date), null, max(redeemed_date)
from stg_voucher_redemptions where redeemed_date is not null
```

No date is hardcoded anywhere in the date dimension. Point the project at a different extract and the window moves with the data. The current window is 1 January 2026 to 31 July 2026, 212 days, derived at build time.

Why it matters beyond this dataset

The brief specified no dates. Whether a redemption tail belongs inside the reporting period is an **unstated requirement** — the kind that produces a number which is wrong and entirely plausible. Those are the errors worth engineering against, because nobody reviewing a dashboard will spot them by eye.

Current state: 28 controls, 27 tie exactly, one documented rounding convention, zero failures.